

OUTPUT MAPPING SPECIFICATION

Sales Call Prep-Sheet Generator Output & Agent Mapping

Exact mapping of OpenRouter models → output fields → Streamlit UI, across a multi-agent chained pipeline

Frontend: Streamlit

Scraping: Jina Reader

LLM access: OpenRouter API (key stored in `.env`)

Architecture: Multiple chained agents — different OpenRouter models per output section

Team size: 2 people, no fixed deadline

Scope of this document: Output mapping only — which agent produces which field, in what shape, and how it lands in the UI

1. Agent Pipeline Overview

The system is a chain of independent, async-callable functions. Each function calls one OpenRouter model with a narrow, single-purpose prompt, and returns one well-defined chunk of the final output. No single agent is responsible for the whole JSON — this keeps prompts short, keeps each model's job easy to verify, and lets you swap models per-stage without touching the rest of the pipeline.



Stage	Responsible for	Consumes	Produces
0. Scrape	Raw context gathering	Company name/URL	Clean markdown text (raw_context)
1. Company Brief Agent	Summarize + extract milestones	raw_context	short_summary , recent_milestones[]
2. Pain Point Agent	Role-specific challenges	raw_context, job_title, industry	strategic_pain_points[]
3. Icebreaker Agent	Open-ended conversation starters	short_summary, milestones, job_title	icebreaker_questions[]
4. Hook / Pitch Agent	The single golden opening line + tailored pitch	ALL prior outputs + seller's product	golden_hook , tailored_pitch

Why this order matters: The Hook/Pitch Agent runs last and is given everything the earlier agents produced, not just the raw scraped text. It is the only agent that needs full context — it's effectively the "closer" that synthesizes the summary, the pain points, and the milestones into one punchy line plus a short tailored pitch paragraph. Running it last means it never has to re-derive things the earlier agents already extracted.

2. Per-Agent Output Mapping

Each agent below maps to one async function in your pipeline. The "OpenRouter model" column gives a concrete suggestion, but since you're using OpenRouter, any of these are swappable by changing one string in your `.env`-driven config — no code changes needed if you keep model names in environment variables.

Agent 1 — Company Brief Agent

Property	Value
Input	<code>raw_context</code> (scraped markdown from Jina Reader)
Suggested OpenRouter model	A fast, cheap model — e.g. <code>openai/gpt-4o-mini</code> or <code>google/gemini-flash-1.5</code> via OpenRouter. This stage is pure extraction/summarization, not creative reasoning, so cost/speed should dominate the choice.
Task	Condense the scraped text into a 2-sentence company summary and extract up to 2 recent, concrete milestones (funding, launches, leadership changes, expansions).

`short_summary`

Type: string

Shape: exactly 2 sentences, plain text, no markdown

UI placement: Top of the output panel, directly under the company name header, in `st.write()` or `st.markdown()` as regular body text

`recent_milestones`

Type: array of strings (0-2 items)

Shape: each item is one short clause, e.g. `"Raised $40M Series B in March 2026"`

UI placement: Rendered as a small bulleted list under a "Recent Milestones" subheader — can double as icebreaker source material for Agent 3

```
{
  "short_summary": "string, 2 sentences",
  "recent_milestones": ["string", "string"]
}
```

Agent 2 — Pain Point Agent

Property	Value
Input	<code>raw_context</code> , <code>job_title</code> , optional <code>industry</code>
Suggested OpenRouter model	A stronger reasoning model — e.g. <code>anthropic/claude-3.5-sonnet</code> or <code>openai/gpt-4o</code> via OpenRouter. This is the highest-value reasoning step in the whole pipeline, so it's worth spending more tokens/cost here than on Agent 1.
Task	Given the company context and the exact job title, infer exactly 3 specific operational pain points that role is likely facing right now — not generic industry pain points, but ones grounded in the scraped company context where possible.

`strategic_pain_points`

Type: array of exactly 3 objects

Shape: each object has `challenge` (string) and `why_it_matters` (one-sentence business impact)

UI placement: 3 stacked cards or `st.expander` blocks under a "Strategic Pain Points" subheader — one card per challenge, with `why_it_matters` as smaller subtext inside each card

```
{
  "strategic_pain_points": [
    {
      "challenge": "string — specific operational headache for this exact role",
      "why_it_matters": "string — 1 sentence on business impact"
    }
    // exactly 3 entries, no more, no fewer
  ]
}
```

Reliability note: Enforce the "exactly 3" constraint in code, not just in the prompt. If the model returns 2 or 4, either re-prompt once or

Agent 3 — Icebreaker Agent

Property	Value
Input	<code>short_summary</code> , <code>recent_milestones</code> , <code>job_title</code>
Suggested OpenRouter model	A mid-tier model — e.g. <code>mistralai/mistral-large</code> or <code>openai/gpt-4o-mini</code> . This task is light reasoning over already-condensed inputs, so it doesn't need the most expensive model.
Task	Generate 2-3 open-ended icebreaker questions a salesperson could genuinely ask this prospect, grounded in the milestones and the role — not generic small talk.

`icebreaker_questions`

- Type:** array of strings (2-3 items)
- Shape:** each item is a single open-ended question ending in "?"
- UI placement:** Numbered list under an "Icebreakers" subheader, styled as a quick-reference list the rep can glance at right before the call

```
{
  "icebreaker_questions": [
    "string ending in ?",
    "string ending in ?"
  ]
}
```

Agent 4 — Hook / Pitch Agent Closer Agent

Property	Value
Input	ALL prior outputs (<code>short_summary</code> , <code>recent_milestones</code> , <code>strategic_pain_points</code>) + <code>job_title</code> + <code>seller_product</code>
Suggested OpenRouter model	Your strongest available model via OpenRouter — e.g. <code>anthropic/claude-3.5-sonnet</code> or <code>openai/gpt-4o</code> . This is the single line and short paragraph the rep will actually read out loud or paste into an email, so language quality matters most here.
Task	Act as an elite B2B sales coach. Write one hyper-personalized opening line connecting the seller's product to the prospect's situation, then a short (3-4 sentence) tailored pitch paragraph that explicitly references at least one of the pain points from Agent 2.

`golden_hook`

- Type:** string
- Shape:** one sentence, no more than ~30 words, written to be said aloud or pasted verbatim into an email opener
- UI placement:** Rendered inside a styled `st.info()` box or a markdown blockquote at the very top of the output panel — this is the single most copy-pasteable element in the whole app, so it should be the most visually prominent

`tailored_pitch`

- Type:** string
- Shape:** 3-4 sentences, references the seller's product by name and at least one specific pain point
- UI placement:** Below the `golden_hook` box, under a "Suggested Pitch" subheader, as a normal paragraph

```
{
  "golden_hook": "string, 1 sentence, <=30 words",
  "tailored_pitch": "string, 3-4 sentences"
}
```

3. Full Assembled Output Contract

After all 4 agents run, your orchestration function merges their outputs into one final JSON object. This is the single object your Streamlit app receives and renders — it never needs to know that it came from 4 separate model calls.

```
{
  "company_name": "string",
  "job_title": "string",
  "seller_product": "string",

  "short_summary": "string",
  "recent_milestones": ["string", "string"],

  "strategic_pain_points": [
    { "challenge": "string", "why_it_matters": "string" },
    { "challenge": "string", "why_it_matters": "string" },
    { "challenge": "string", "why_it_matters": "string" }
  ],

  "icebreaker_questions": ["string", "string"],

  "golden_hook": "string",
  "tailored_pitch": "string",

  "meta": {
    "data_source": "live" ,
    "generated_at": "ISO 8601 timestamp"
  }
}
```

The "data_source" field: Set this to "live" when Jina Reader successfully scraped real content, or "cached" if you fall back to a saved mock dataset. Surface this honestly in a small UI badge — it costs nothing to build and signals engineering maturity if anyone asks whether the data is real-time.

3.1 Streamlit UI Layout Mapping

UI Section (top → bottom)	Field(s) rendered	Streamlit element
Header	company_name , job_title	st.title() / st.subheader()
Golden hook box	golden_hook	st.info() or styled blockquote via st.markdown()
Company brief	short_summary	st.write()
Milestones	recent_milestones	st.markdown() bullet list
Pain points	strategic_pain_points	3x st.expander() or styled cards via st.container(border=True)
Icebreakers	icebreaker_questions	Numbered st.markdown() list
Suggested pitch	tailored_pitch	st.write() under subheader
Footer / meta	meta.data_source , meta.generated_at	Small st.caption() at the bottom

4. Orchestration Pseudocode

This shows how the 4 agents chain together as async functions, and how each one's model is configured independently via OpenRouter + .env .

```
.env
-----
OPENROUTER_API_KEY=sk-or-xxxxxxxxxxxxx
MODEL_COMPANY_BRIEF=google/gemini-flash-1.5
MODEL_PAIN_POINTS=anthropic/claude-3.5-sonnet
MODEL_ICEBREAKERS=openai/gpt-4o-mini
MODEL_HOOK_PITCH=anthropic/claude-3.5-sonnet
```

```
orchestrator.py (pseudocode)
-----
async def generate_prep_sheet(company, job_title, seller_product):

    raw_context = await fetch_via_jina_reader(company)    # Stage 0

    brief = await call_openrouter(
        model=os.getenv("MODEL_COMPANY_BRIEF"),
```

```

    prompt=build_brief_prompt(raw_context)
) # -> { short_summary, recent_milestones }

pain_points = await call_openrouter(
    model=os.getenv("MODEL_PAIN_POINTS"),
    prompt=build_pain_point_prompt(raw_context, job_title)
) # -> { strategic_pain_points }

icebreakers = await call_openrouter(
    model=os.getenv("MODEL_ICEBREAKERS"),
    prompt=build_icebreaker_prompt(brief, job_title)
) # -> { icebreaker_questions }

hook_pitch = await call_openrouter(
    model=os.getenv("MODEL_HOOK_PITCH"),
    prompt=build_hook_prompt(brief, pain_points, job_title, seller_product)
) # -> { golden_hook, tailored_pitch }

return merge_all(company, job_title, seller_product,
                  brief, pain_points, icebreakers, hook_pitch)

```

Why .env-driven model selection matters here: Because you're going through OpenRouter, every agent's model is just a string. Keeping `MODEL_PAIN_POINTS`, `MODEL_HOOK_PITCH`, etc. as separate environment variables (rather than hardcoding model names in each prompt function) means you and your teammate can A/B different models per-stage without redeploying — e.g. testing whether a cheaper model is "good enough" for the Icebreaker Agent while keeping a stronger model on the Pain Point and Hook agents, which are the two stages that actually carry the product's perceived intelligence.

4.1 Suggested `call_openrouter()` Contract

```

async def call_openrouter(model: str, prompt: str) -> dict:
    """
    Sends prompt to OpenRouter, requests JSON-only output,
    parses and validates against the expected schema for that stage.
    Raises/falls back if parsing fails — never passes raw text upward.
    """
    response = await openrouter_client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object"}
    )
    return validate_and_parse(response, expected_schema_for(model))

```

Validation is mandatory at every stage: Since 4 different models are involved, don't assume all of them respect the JSON-only instruction equally well. Validate each agent's output against its expected schema immediately after the call, before passing it into the next agent's prompt — a malformed Agent 1 output will otherwise silently corrupt Agents 3 and 4 downstream.

Output mapping specification — companion to the full build roadmap document.